

0.22. Remonter : `deplace_y` et un pas négatif

Le 0.21, plus une ligne : `-pas` fait remonter la lettre. Et pourquoi deux fonctions, une par axe.

1. Ce qu'il faut comprendre

C'est le 0.21, plus une ligne : la montée.

Ce qui est nouveau ici : le pas négatif **sur Y**. `-pas` fait **monter** : les numéros de ligne diminuent vers le haut. Comme au 0.19, `-5` est rangé 251, et `deplace_y` lit tout ce qui dépasse 127 comme une montée.

La montée part de `y`, qui vaut 5 grâce au `y =`. Elle rend 0, et `y =` le range.

Pourquoi deux fonctions, et pas une seule pour X et Y ? Une fonction ne peut rendre qu'une seule valeur.

`deplace_x` rend la colonne, `deplace_y` rend la ligne : chacune dit exactement où la lettre s'est arrêtée.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_y>` → [0.20.3. La fonction `deplace\_y\(\)` — avancer sur une colonne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1092 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage – la mise en route de la console
deplace_y()    158 o ← main() – une fonction fournie par le langage
main()         49 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 2   26 o sur place, dans deplace_y() + AdresseCase (10 o)
effacer()    x 1   15 o sur place, dans deplace_y() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 2    6 o sur place, dans deplace_y(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_y() x 2   38 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_y> // fait avancer une tuile sur sa colonne
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5; // +5 : descend ; -5 : monte
```

3. UNE VARIABLE — Y

```
uint8_t y = 0; // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    y = deplace_y(0, y, ALPHABET[0], pas); // la descente (le 0.21) : y vaut 5

    // La ligne ajoutée : le même appel avec -pas, donc vers le HAUT.
    // Elle part de y (5) et rend 0 ; « y = » range ce 0 dans y.
    y = deplace_y(0, y, ALPHABET[0], -pas); // après cette ligne : y vaut 0

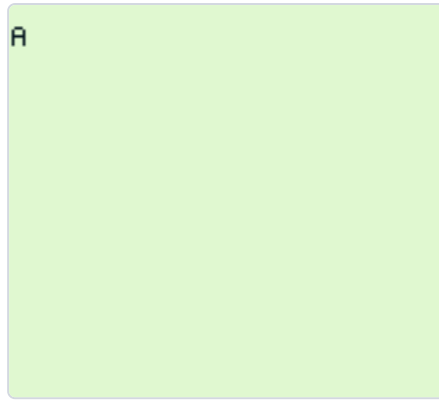
    while (true) {
        image(); // la lettre est revenue tout en haut
    }
}
```

3. Ce qui se passe, étape par étape

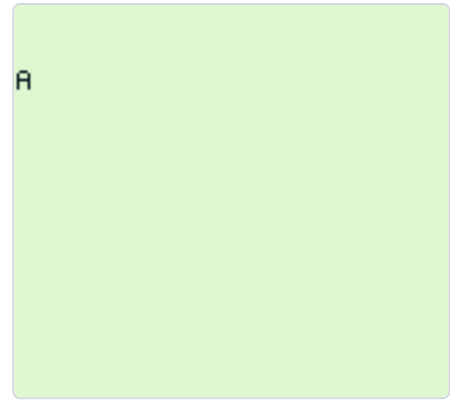
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



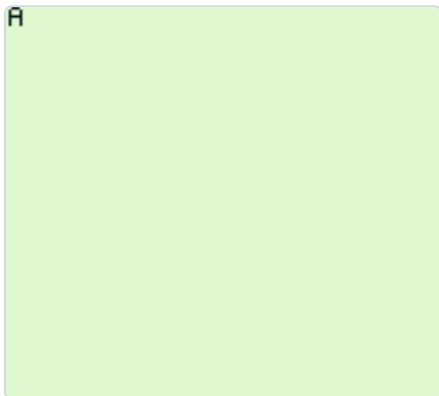
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend de 5 lignes, puis remonte de 5 lignes, et s'arrête tout en haut.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à la descente, le A arrive en ligne 5 (« A »)

✓ à la montée, le A revient en ligne 0 (« A »)

✓ y vaut la ligne rendue ($y = 0$)

Ensuite : **0.22.1. Remonter : `deplace_y` et un pas négatif — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++